MMDU MULLANA OFFICIAL SYLLABUS • BCSE-007 • UNIT 2

Unit 2: Stacks, Expressions (Infix/Postfix/Prefix) & Recursion (Tower of Hanoi)

Subject: Data Structures & Algorithms (BCSE-007) • **Exam Weightage:** 25 Marks • **Study Time:** 35 mins

🗺️ UNIT 2 SYLLABUS ROADMAP & TOPIC INDEX (TABLE OF CONTENTS)

3 Modules • 10 Topics • Pages 1–3

1. 1. Stack ADT & Primitive Operations — Pg. 1

- TOPIC 1. Definition & Abstract Data Type (ADT) — Pg. 1
- TOPIC 2. Primitive Operations on Array-Based Stack — Pg. 1

2. 2. Expression Conversion (Shunting-Yard) & Evaluation — Pg. 1–2

- TOPIC 1. Infix, Postfix and Prefix Notations — Pg. 1
- TOPIC 2. Operator Precedence & Associativity — Pg. 1
- TOPIC 3. Algorithm: Infix to Postfix (Shunting-Yard) — Pg. 2
- TOPIC 4. Algorithm: Postfix Expression Evaluation — Pg. 2

3. 3. Recursion & The Tower of Hanoi Problem — Pg. 2–3

- TOPIC 1. Recursion Mechanics & System Call Stack — Pg. 2
- TOPIC 2. Tower of Hanoi — Pg. 2
- TOPIC Recursive Algorithm: — Pg. 3
- TOPIC Recurrence Relation & Solution: — Pg. 3

1. Stack ADT & Primitive Operations

1. Definition & Abstract Data Type (ADT)

A **Stack** is a linear, restricted list in which additions (push) and deletions (pop) take place exclusively at a single designated end called the **Top**. It strictly enforces **LIFO (Last-In, First-Out)** order.

2. Primitive Operations on Array-Based Stack

```c
#define MAX 100
int stack[MAX], top = -1;

void push(int val) {
    if (top == MAX - 1) {
        printf("Stack Overflow!
");
        return;
    }
    stack[++top] = val; // Increment top, then store value
}

int pop() {
    if (top == -1) {
        printf("Stack Underflow!
");
        return -1;
    }
    return stack[top--]; // Retrieve value, then decrement top
}

int peek() {
    if (top == -1) return -1;
    return stack[top]; // O(1) query without removal
}
```

2. Expression Conversion (Shunting-Yard) & Evaluation

1. Infix, Postfix and Prefix Notations

- **Infix:** Operator placed between operands (human standard): A + B *C*

- ***Postfix (Reverse Polish Notation):*** *Operator follows operands: A B C +*

- **Prefix (Polish Notation):** Operator precedes operands: + A * B C

2. Operator Precedence & Associativity

1. **Parentheses (), []:** Evaluated first

2. **Exponentiation ^:** Highest precedence, **Right-to-Left** associativity

3. **Multiplication & Division *, /, %: **High precedence,** Left-to-Right

4. Addition & Subtraction +, -: **Lowest precedence,** Left-to-Right

3. Algorithm: Infix to Postfix (Shunting-Yard)

1. **Initialize an empty operator stack.**

2. **Scan infix token by token from Left to Right:**

- Operand: **Append directly to output postfix expression.**
- Left Parenthesis (: **Push onto stack.**
- Right Parenthesis): **Pop operators and append to output until (is encountered; discard both parentheses.**
- Operator op: **While stack is not empty and precedence(stack[top]) >= precedence(op) (strictly > if op is right-associative), pop and append to output. Push op onto stack.**

3. **Pop and append all remaining operators from stack to output.**

4. Algorithm: Postfix Expression Evaluation

1. **Scan postfix expression from Left to Right.**

2.**If token is an** operand **, push onto operand stack.**

3.**If token is an** operator**:

- Pop op2 = pop() (Second operand popped first!).
- Pop op1 = pop() (First operand popped second!).
- Calculate result = op1 [operator] op2.
- Push result back onto stack.

4. When scan completes, the single value remaining on the stack is the final expression value.

3. Recursion & The Tower of Hanoi Problem

1. Recursion Mechanics & System Call Stack

Recursion occurs when a function invokes itself directly or indirectly. The runtime system allocates an **Activation Record (Stack Frame)** on the call stack containing:

1. Actual parameters

2. Local variables

3. Return memory address to the calling routine

Omission of a valid **base case** causes unbounded recursive expansion, exhausting available stack memory and triggering a runtime **Stack Overflow**.

2. Tower of Hanoi

Problem Statement: Move n disks from Source Peg S to Destination Peg D using Auxiliary Peg A, subject to:

1. Only one disk can be moved at a time.

2. A larger disk may never be placed on top of a smaller disk.

Recursive Algorithm:

```
void towerOfHanoi(int n, char S, char A, char D) {
    if (n == 1) {
        printf("Move disk 1 from %c to %c
", S, D);
        return;
    }
    towerOfHanoi(n - 1, S, D, A); // Step 1: Move n-1 disks from S to A using D
    printf("Move disk %d from %c to %c
", n, S, D); // Step 2: Move disk n to D
    towerOfHanoi(n - 1 A, S, D); // Step 3: Move n-1 disks from A to D using S
}
```

Recurrence Relation & Solution:

$T(n) = 2 \cdot T(n-1) + 1$ with $T(1) = 1$
Using repeated backward substitution:
$T(n) = 2^1 \cdot T(n-1) + 1 = 2^2 \cdot T(n-2) + 2 + 1 = \ldots = \Sigma_{i=0}^{n-1} 2^i = 2^n - 1$
Total moves required: $2^n - 1$. Time complexity is strictly $\Theta(2^n)$.